

- **Java Core là phần kiến thức nền tảng của ngôn ngữ Java.**
- **Java 8 là một phiên bản (version) của Java (2014)**
 - Lambda Expression
 - Functional Interface
 - Method Reference
 - Stream API

Lambda Expression (chỉ dành cho Interface chỉ có 1 phương thức trừu tượng)

```
Collections.sort(list, new Comparator<Student>() {  
    @Override  
    public int compare(Student a, Student b) {  
        return a.getAge() - b.getAge();  
    }  
});
```

Java 8

```
Collections.sort(list,  
    (a, b) -> a.getAge() - b.getAge());
```

Functional Interface

Java Core

```
public interface Calcul {  
    int add(int a, int b);  
}  
  
public class Test1 {  
    public static void main(String[] args) {  
        Calcul c = new Calcul() {  
            @Override  
            public int add(int a, int b) {  
                return a+b;  
            }  
        };  
        System.out.println("c.add(a: 3, b: 4));  
    }  
}
```

Java 8

```
interface Calculator {  
    int add(int a, int b);  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Calculator c = (a, b) -> a + b;  
  
        System.out.println(c.add(3, 5));  
  
    }  
}
```

Với hàm không tham số

```
interface Hello {  
  
    void sayHello();  
  
}
```

```
Hello h = () -> System.out.println("Xin chào Java 8");  
  
h.sayHello();
```

```
public interface Calcul {  
    int add(int a, int b);  
}
```

```
public static void execute(String s, int a, int b, Calculator c) {  
    System.out.println(s+ c.calcul(a, b));  
}
```

```
execute(s: "Cong :", a: 5, b: 3, (x, y) -> x + y);  
execute(s: "Tru :", a: 5, b: 3, (x, y) -> x - y);  
execute(s: "Nhan :", a: 5, b: 3, (x, y) -> x * y);  
execute(s: "Chia :", a: 5, b: 3, (x, y) -> x / y);
```

Cong :8

Tru :2

Nhan :15

Chia :1

Method Reference

Method Reference là cách viết ngắn gọn của Lambda khi Lambda chỉ gọi một phương thức có sẵn

Cú pháp: **TênLớpHoặcĐốiTượng::TênPhươngThức**

```
ArrayList<String> list = new ArrayList<>();
list.add(e: "Java ");
list.add(e: "Python ");
list.add(e: "C++");
// cach 1
for (String x: list)
    System.out.print(s: x);
System.out.println();
// cach 2 : Lamda
list.forEach(x -> System.out.print(s: x));
// Method Reference
System.out.println();
list.forEach(System.out::print);
System.out.println();
,
```

```
List<Integer> list = Arrays.asList(5, 8, 2, 9, 1);
list.sort((a, b) -> a.compareTo(b)); // Lamda
//list.sort(Integer::compareTo); //Method reference
System.out.println(list);
```

Stream API

- Stream API là một đường ống (Pipeline) xử lý dữ liệu lấy từ một nguồn dữ liệu (Collection, Array, File...) mà **không làm thay đổi dữ liệu gốc**.

```
List<Integer> list = Arrays.asList(5, 8, 2, 9, 1);  
for(Integer x : list)  
    if(x % 2 == 0)  
        System.out.println(x);  
  
// dùng stream  
list.stream().filter(x -> x % 2 == 0).forEach(System.out::println);
```

- Stream gồm 3 thành phần
 - Nguồn (Source) : collection, Array, File, Set, Map
 - Thao tác trung gian (Intermediate Operations): filter, map, sorted,...
 - Thao tác kết thúc (Terminal Operation) : foreach(..), collect(...), count(), max(), min()

Bảng tóm tắt phương thức hay dùng

Nhóm	Phương thức	Chức năng
Tạo Stream	<code>stream()</code>	Tạo luồng xử lý từ Collection
Lọc	<code>filter()</code>	Giữ các phần tử thỏa điều kiện
Biến đổi	<code>map()</code>	Chuyển đổi dữ liệu
Sắp xếp	<code>sorted()</code>	Sắp xếp dữ liệu
Loại trùng	<code>distinct()</code>	Loại bỏ phần tử trùng
Giới hạn	<code>limit()</code>	Lấy <code>n</code> phần tử đầu
Bỏ qua	<code>skip()</code>	Bỏ qua <code>n</code> phần tử đầu
Thống kê	<code>count()</code>	Đếm số phần tử
Tìm kiếm	<code>max()</code> , <code>min()</code>	Tìm phần tử lớn nhất, nhỏ nhất
Thu kết quả	<code>collect()</code>	Chuyển về <code>List</code> , <code>Set</code> , <code>Map</code>
Duyệt	<code>forEach()</code>	Thực hiện hành động trên từng phần tử

Các thao tác phổ biến(1)

filter()

Lọc dữ liệu.

`</>` Java

```
.filter(x -> x % 2 == 0)
```

2 5 8 9 10

↓

2 8 10

map()

Biến đổi dữ liệu.

`</>` Java

```
.map(x -> x * 2)
```

2 5 8

↓

4 10 16

Các thao tác phổ biến(2)

sorted()

Sắp xếp.

```
</> Java  
  
.sorted()
```

Hoặc

```
</> Java  
  
.sorted(Comparator.reverseOrder())
```

distinct()

Loại bỏ phần tử trùng.

```
</> Java  
  
.distinct()
```

limit()

Lấy n phần tử đầu.

```
</> Java  
  
.limit(3)
```

Các thao tác phổ biến(3)

skip()

Bỏ qua n phần tử đầu.

```
</> Java
```

```
.skip(2)
```

max()

Lấy phần tử lớn nhất.

```
</> Java
```

```
.max(Integer::compareTo)
```

collect()

Thu kết quả về Collection.

```
</> Java
```

```
.collect(Collectors.toList())
```

count()

Đếm số phần tử.

```
</> Java
```

```
.count()
```

min()

Lấy phần tử nhỏ nhất.

```
</> Java
```

```
.min(Integer::compareTo)
```

forEach()

Duyệt từng phần tử.

```
</> Java
```

```
.forEach(System.out::println)
```

Ví dụ

- Viết chương trình tạo ngẫu nhiên n số nguyên; sau đó thực hiện các công việc sau: liệt kê, tổng các số chẵn, liệt kê số nguyên tố, sắp xếp giảm dần

```
static ArrayList<Integer> list = new ArrayList<>();
```

```
public static void printList() {  
    System.out.println("Danh sách:");  
    for (Integer x : list) {  
        System.out.print(x + " ");  
    }  
    System.out.println();  
}
```

```
public static void printList() {  
    System.out.println("Danh sách:");  
    list.forEach(x -> System.out.print(x + " "));  
    System.out.println();  
}
```

```
public static void sumEven() {
    int sum = 0;
    for (Integer x : list) {
        if (x % 2 == 0) sum += x;
    }
    System.out.println("Tổng chẵn = " + sum);
}

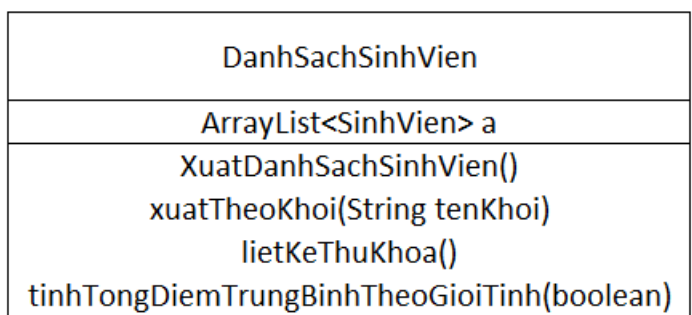
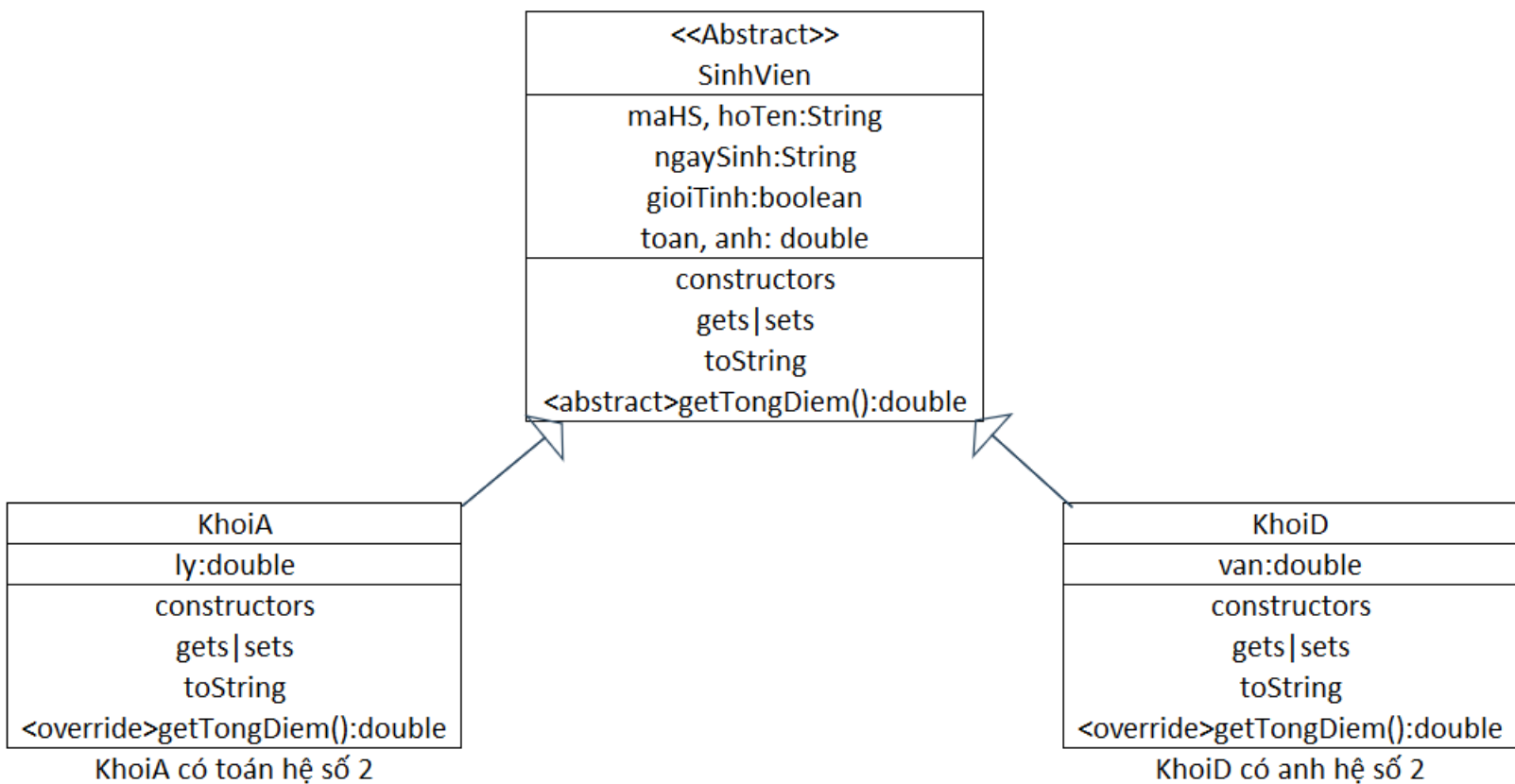
public static void printPrime() {
    System.out.println("Các số nguyên tố:");
    for (Integer x : list) {
        if (isPrime(x))
            System.out.print(x + " ");
    }
    System.out.println();
}

public static void sortDesc() {
    Collections.sort(list, Collections.reverseOrder());
    System.out.println("Danh sách giảm dần:");
    printList();
}
```

```
public static void sumEven() {
    int sum = list.stream()
        .filter(x -> x % 2 == 0)
        .mapToInt(Integer::intValue)
        .sum();
    System.out.println("Tổng chẵn = " + sum);
}

public static void printPrime() {
    System.out.println("Các số nguyên tố:");
    list.stream()
        .filter(x -> isPrime(x))
        .forEach(x -> System.out.print(x + " "));
    System.out.println();
}

public static void sortDesc() {
    // Collections.sort(list, Collections.reverseOrder());
    // System.out.println("Danh sách giảm dần:");
    // printList();
    list = list.stream()
        .sorted(Comparator.reverseOrder())
        .collect(Collectors.toCollection(ArrayList::new));
    printList();
}
```



```

public void xuấtDanhSachSinhVien() {
    a.forEach(System.out::println);
}

public void xuấtTheoKhoi(String tenKhoi) {
    if (tenKhoi.equalsIgnoreCase("A")) {
        a.stream()
            .filter(x -> x instanceof KhoiA)
            .forEach(System.out::println);
    } else {
        a.stream()
            .filter(x -> x instanceof KhoiD)
            .forEach(System.out::println);
    }
}
}

```

```

public void lietKeThuKhoa() {
    double max = a.stream()
        .mapToDouble(SinhVien::getTongDiem)
        .max()
        .orElse(0);
    a.stream()
        .filter(x -> x.getTongDiem() == max)
        .forEach(System.out::println);
}

public double tinhTongDiemTrungBinhTheoGioiTinh(boolean gt) {
    return a.stream()
        .filter(x -> x.isGioiTinh() == gt)
        .mapToDouble(SinhVien::getTongDiem)
        .average()
        .orElse(0);
}
}

```

sapXepTheoTongDiemGiamDan() => a.sorted(Comparator.comparing(SinhVien::getTongDiem).reversed())

lietKeHocLucGioi() => lọc theo getTongDiem()

thongKeSoLuongTheoKhoi() => Collectors.counting()

timSinhVienCoDiemToanCaoNhat() => max(Comparator.comparing(SinhVien::getToan))